# Recall.AI: A Source-Grounded Multi-Agent Organizational Memory System with Decision-Centric Graph Retrieval

Atharva
Independent Researcher
Pune, India
Author contact withheld for artifact preparation

Abstract—Enterprise knowledge is distributed across documents, meeting records, spreadsheets, images, audio, and collaboration systems. Keyword search recovers lexical matches but does not preserve why a decision was made, who participated, what alternatives were rejected, or what downstream impact may follow. Retrieval-augmented generation improves access to unstructured evidence, yet text-only retrieval is weak when the answer depends on relationships among people, decisions, reasons, and projects. This paper presents Recall.AI, an implemented organizational-memory platform that combines multi-source ingestion, structured decision extraction, a Neo4j property graph, Chroma vector retrieval, and LangGraph/LangChain agents. A router classifies a question as historical query or impact analysis; specialized agents invoke graph and raw-memory tools under project and source constraints, and the response includes a trace of the evidence-retrieval actions. The system supports PDF, image, spreadsheet, and audio inputs, with Slack and Microsoft Teams integration points, FastAPI services, a Next.js interface, Supabase application services, and selectable Groq or local Ollama language-model providers. We formalize the graph, vector, and hybrid retrieval objectives and specify an evaluation protocol covering retrieval, answer faithfulness, source attribution, latency, and resource use. Because the repository does not contain a controlled benchmark or measured experiment log, this paper reports implementation evidence and reproducible evaluation procedures rather than invented numerical results. The contribution is a modular, source-grounded architecture for decision memory and impact-oriented question answering, together with explicit safeguards for scope filtering, provenance, and deployment limitations.

Index Terms—organizational memory, knowledge graph, GraphRAG, retrieval-augmented generation, multi-agent systems, decision intelligence, source attribution, enterprise search

## I. Introduction

Organizations accumulate knowledge faster than they can curate it. A decision may first appear in a meeting, be justified in a PDF, refined in a spreadsheet, discussed in a chat channel, and operationalized by a different team. The resulting evidence is fragmented across incompatible representations and access boundaries. When an employee changes role or leaves the organization, the information may remain technically stored but become practically inaccessible: the organization has retained artifacts but lost the decision context that connects them.

Traditional enterprise search addresses only part of this problem. Lexical retrieval is effective when a user knows the exact terminology used in the source, but it is brittle under paraphrase and cannot directly answer relational questions such as who approved a migration, which alternatives were considered, or what other decisions depend on a component. Neural retrieval improves semantic matching but still tends to return independent passages. A conventional retrieval-augmented generation (RAG) pipeline then asks a language model to reconstruct relationships from a small set of text chunks. That reconstruction can be useful, but it is difficult to audit and may omit the edges that make an organizational explanation meaningful.

Knowledge graphs provide a complementary representation. A decision can be connected to its topic, reasons, alternatives, people, source, project, and timestamp. Graph traversal makes those relationships explicit, while vector search retains the unstructured evidence needed to preserve nuance. Recent GraphRAG studies show that graph construction, indexing, retrieval, and generation are separate design choices rather than a single algorithm [1]–[3]. Enterprise use-case research likewise emphasizes that knowledge-graph construction and maintenance must be considered together with the downstream question-answering task [4], [5]. Multi-agent KGQA work further motivates separating routing, entity and relationship retrieval, and answer synthesis [6].

This paper presents Recall.AI, a working software system whose central abstraction is organizational decision memory. The implementation ingests several source types, extracts structured knowledge using schema-constrained language-model calls, persists decision projections in Neo4j, indexes raw text in Chroma, and exposes source-grounded query and impact-analysis agents through a FastAPI service and Next.js client. The system is designed around a conservative answer policy: agents must answer from tool results, respect an exact source filter when present, and return source traces. The architecture therefore treats provenance as a first-class output rather than a post-processing annotation.

The paper makes the following contributions:

1) A decision-centric organizational-memory architec-

ture that combines property-graph structure with raw-memory vector retrieval.
2) A multi-agent query design that separates historical questions from "what-if" impact questions and provides distinct tool affordances.
3) A multi-source ingestion pipeline covering PDFs, images, spreadsheets, audio, meeting text, and collaboration-system integration points.
4) A source- and project-scoped retrieval policy that constrains both tool arguments and final answer synthesis.
5) Formal definitions for graph retrieval, vector retrieval, score fusion, confidence, and source-trace completeness.
6) A reproducible evaluation protocol that distinguishes implemented behavior from results that require a controlled dataset and measurement run.

## II. Background and Related Work

### A. RAG and GraphRAG

RAG combines a parametric generator with a non-parametric store [7]. Dense retrieval is robust to vocabulary variation, but chunk-level retrieval does not inherently encode multi-hop dependencies. GraphRAG augments the retrieval unit with nodes, edges, communities, or subgraphs. Surveys classify systems by graph type, indexed components, retrieval primitive, and retrieval granularity [2]. The recent unified analysis by Zhou et al. demonstrates that graph construction, index construction, operator configuration, and retrieval/generation must be evaluated separately [1]. Recall.AIadopts a deliberately lightweight property graph rather than a community-detection pipeline: the graph is centered on decisions and their evidence-bearing relations, while Chroma preserves text chunks for semantic recall.

### B. Knowledge-graph question answering

Knowledge-graph question answering traditionally maps natural-language questions to structured queries, including SPARQL or Cypher. The difficulty is not only query generation but entity and relation resolution under open-ended user language. SCIGENT addresses this by separating end-to-end question answering into cooperating agents and evaluates entity, query, and end-to-end behavior [6]. Recall.AIshares the separation principle but uses bounded LangChain tools over application-owned Neo4j and Chroma stores. This avoids exposing arbitrary database-query generation to the user-facing agent and makes the evidence path inspectable.

### C. Enterprise knowledge management and decision support

Enterprise KG research identifies knowledge acquisition, maintenance, reasoning, and governance as coupled concerns [4], [5]. Industrial risk-analysis work shows how graph-enhanced RAG can connect structured failure information to textual evidence and improve reasoning over effects and causes [8]. These observations motivate a decision-centric schema in which a reason, alternative, impact, and responsible person are stored as explicit neighbors rather than left only inside a passage.

TABLE I
Positioning of organizational-memory approaches.

| Approach | Semantic retrieval | Relationship reasoning | Traceability | Main limitation |
|---|---|---|---|---|
| Keyword search | Low | No | Source link | Vocabulary and silo dependence |
| Vanilla RAG | High | Implicit | Passage context | Relations reconstructed by generator |
| GraphRAG | High | Explicit | Graph evidence | Graph construction and freshness |
| Recall.AI | High | Decision-centered | Tool/source trace | No temporal conflict model yet |

### D. Temporal and provenance concerns

Knowledge in organizations changes. A decision may be superseded, and a source may represent a plan rather than an executed action. Temporal GraphRAG research identifies temporal ambiguity and redundancy as causes of retrieval failure [9]. The present implementation stores an optional timestamp and source on decision nodes but does not yet implement temporal validity intervals or conflict resolution. We therefore treat temporal GraphRAG, version-aware edges, and conflict detection as future work rather than claiming capabilities not present in the repository.

## III. Problem Formulation

Let an organization contain a set of sources $S$, documents and transcripts $D$, and a decision graph $G = (V, E)$. A decision record is represented by the tuple

$$d = (a, t, r, i, A, P, \tau, s, p), \tag{1}$$

where $a$ is the action, $t$ the topic, $r$ the reason, $i$ the impact, $A$ alternatives, $P$ people, $\tau$ an optional timestamp, $s$ the source, and $p$ the project. The system must answer two classes of question: historical query $q_h$ (what, why, who, when) and impact query $q_i$ (what changes, breaks, or is affected if a decision changes).

The design objective is not to maximize a single language-model score. It is to return an answer $y$ that

is relevant to $q$, supported by retrieved evidence $R(q)$, scoped to the requested source/project, and accompanied by a trace $T(q)$. We write:

$$y = \mathrm{Gen}(q, R_g(q), R_v(q), T(q)), \tag{2}$$

where $R_g$ is structured graph evidence and $R_v$ is raw-memory evidence. A valid answer satisfies $\mathrm{scope}(R, q) = 1$ and $\mathrm{support}(y, R) = 1$ under the system's answer policy. These predicates are evaluated by human or automated annotation in the proposed benchmark; they are not assumed to be guaranteed by the language model.

## IV. System Architecture

Figure 1 shows the implemented logical architecture. The browser client is a Next.js/React application with authentication, project, graph, query, activity, file, and team views. FastAPI routes mediate upload, query, graph, project, activity, health, and collaboration operations. Application services isolate persistence and authorization concerns from ingestion and agent orchestration.

The ingestion path detects the file extension and dispatches to PyMuPDF for PDF text, OpenPyXL-based extraction for spreadsheets, image extraction for raster inputs, and audio transcription for supported audio/video formats. Meeting text has a dedicated extraction schema with categories such as decision, action item, risk, requirement, participant, deadline, and technology. Generic content is chunked, passed through schema-constrained extraction, and projected into Neo4j; raw content is stored as embeddings in Chroma when the vector path is enabled.

The query path first routes the question to QUERY or IMPACT. The query agent uses structured decision search and raw-memory search. The impact agent uses related-decision search, person-based search, and raw-memory search. Both agents enforce optional *source_filter* and *project_id* parameters and return an answer, a textual reasoning summary of tools used, and a source trace containing tool names, arguments, and result previews.

### A. Knowledge graph model

The core graph uses Organization and Project containers; Decision nodes connect to Person through MADE_BY, Reason through BASED_ON, Alternative through ALTERNATIVE, and Project through BELONGS_TO. Meeting ingestion adds Meeting and MeetingKnowledge nodes, with HAS_KNOWLEDGE and INVOLVES relations. Source, impact, timestamp, project, and organization are properties on decision or knowledge nodes. Figure 2 abstracts the model without implying node types not present in the code.

### B. Vector memory and hybrid retrieval

Chroma stores overlapping text chunks with source and optional project metadata. Each chunk is embedded using the configured Cohere embedding model. At query time, the vector tool computes a query embedding and requests the nearest chunks, then performs a second hard-filter check so a result with an incorrect source or project is discarded. Neo4j search uses case-insensitive word containment over decision, subject, reason, and person fields. The system therefore combines high-precision structured retrieval with semantically broad raw-memory retrieval, but the current implementation does not expose a learned fusion weight or a benchmark-calibrated ranker.

### C. Model providers and application boundary

Groq is the default cloud model provider with Llama 3.3 70B Versatile; Ollama can be selected for local execution and falls back to Groq if unreachable. Supabase services support application-level authentication and project/team workflows. The provider factory is intentionally isolated from agents so a deployment can change model locality without changing graph or vector tools. This separation is useful for privacy analysis, although it does not by itself guarantee that no sensitive data leaves the deployment.

## V. Ingestion and Knowledge Construction

### A. Multimodal ingestion

The input dispatcher accepts PDF, XLS/XLSX, PNG/JPG/GIF/WEBP, and common audio/video extensions. PDF extraction uses page text from PyMuPDF. Spreadsheet extraction is delegated to an Excel utility. Image extraction and audio transcription are separate services, allowing deployments to replace the provider or add OCR/transcription controls. Slack ingestion and Microsoft Teams integration are represented by dedicated modules; their operational completeness depends on credentials and upstream API availability.

### B. Structured extraction

Generic extraction returns a list of decision items with fields decision, reason, impact, alternatives, people, timestamp, and topic. A Pydantic schema is passed to the model through structured output. Long input is split by paragraphs with a provider-dependent maximum chunk length; each successful chunk result contributes only items with non-empty decisions. Meeting extraction uses a richer schema and stores the raw meeting text in Chroma before persisting structured items.

### C. Graph construction and evidence preservation

Neo4j persistence uses MERGE for organization, project, person, alternative, and reason nodes, while each extracted decision receives a unique identifier. The raw source is retained separately in Chroma so that a structured record can be checked against the underlying text. The two projections are not transactionally coupled in the legacy ingestion path; application services can disable the projections and commit an authoritative transaction before writing them. This is an important boundary for deployment: a successful graph write does not prove that every vector write succeeded, and a vector duplicate is treated as a no-op by the Chroma adapter.

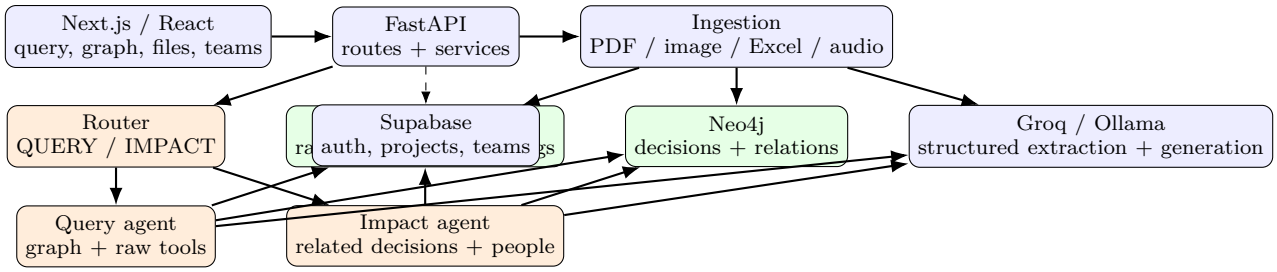


Fig. 1. Logical architecture of Recall.AI. Solid arrows are implemented data paths; dashed arrows denote integration or provider selection points.

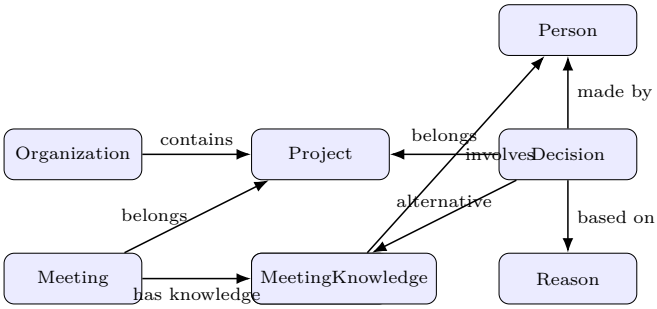


Fig. 2. Decision-centric knowledge-graph schema.

**Algorithm 1** Document ingestion and graph projection

Require: bytes $b$, filename $f$, source $s$, provider $m$

$x \leftarrow \text{Decode}(b, \text{suffix}(f))$
$C \leftarrow \text{Chunk}(x)$
$I \leftarrow \emptyset$
for all $c \in C$ do
  $I \leftarrow I \cup \text{ExtractDecision}(c, m)$
end for
for all $d \in I$ do
  $\text{Neo4jCreate}(d, s, project, organization)$
end for
$\text{ChromaAdd}(\text{OverlapChunks}(x), s, project)$
return $I$

## VI. Multi-Agent Query Processing

The router prompts the selected LLM to emit one of two labels. IMPACT is selected when the question asks what would happen, what breaks, what changes, or what risks arise. All other questions are routed to QUERY. This coarse classification is intentionally simple and transparent; it is not a learned intent model and can misclassify ambiguous questions.

The QUERY agent prioritizes *search_decisions* and uses *search_raw_memory* for additional context or when structured search returns nothing. The IMPACT agent searches related decisions, optionally searches by person, and retrieves raw context. Both use a bounded tool loop; local Ollama mode uses direct tool execution and feeds the resulting context to the base model. Tool-call errors can trigger a direct-execution fallback. The final prompt

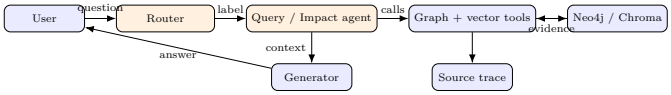


Fig. 3. Query routing, retrieval, reasoning, and source-trace emission.

**Algorithm 2** Scoped hybrid query

Require: question $q$, optional source $s$, project $p$

$k \leftarrow \text{Route}(q)$
if $k = \text{Query}$ then
  $G \leftarrow \text{NeoSearch}(q, s, p)$
else
  $G \leftarrow \text{ImpactSearch}(q, s, p)$
end if
$V \leftarrow \text{VectorSearch}(q, s, p)$
$R \leftarrow \text{Filter}(G \cup V, s, p)$
$y \leftarrow \text{Generate}(q, R, \text{Trace}(G, V))$
return $(y, R, \text{Trace})$

prohibits general knowledge, raw transcript dumping, and mixing sources under an active filter.

## VII. Formal Retrieval and Attribution Model

Let a text chunk embedding be $\mathbf{e}_d \in \mathbb{R}^n$ and the query embedding be $\mathbf{e}_q$. Cosine similarity is

$$\cos(q, d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_d\|_2}. \tag{3}$$

For a graph candidate $v$, define a normalized graph score

$$g(q, v) = w_t \,\text{match}(q, topic_v) + w_a \,\text{match}(q, action_v) + w_r \,\text{match}(q, reas \tag{4}$$

where the weights are an evaluation parameter, not a value hard-coded by the current implementation. A general hybrid ranker can be expressed as

$$H(q, z) = \lambda\, \widetilde{g}(q, z) + (1 - \lambda)\, \widetilde{v}(q, z), \quad 0 \le \lambda \le 1, \tag{5}$$

with min–max or reciprocal-rank normalization. In the current system, graph and vector tools are invoked as separate evidence channels and the LLM receives their results; $H$ is therefore a design formalization for controlled evaluation rather than a claim about a learned production ranker.

TABLE II
Implemented technology stack evidenced by the repository.

| Layer | Components |
|---|---|
| Client | Next.js, React, TypeScript, Tailwind, Framer Motion, GSAP |
| API | FastAPI, Uvicorn, Pydantic settings, multipart uploads |
| Agents | LangChain, LangGraph, Groq Llama 3.3, Ollama fallback |
| Structured store | Neo4j property graph, Cypher |
| Vector store | Chroma Cloud/client, Cohere embeddings |
| Ingestion | PyMuPDF, OpenPyXL, image extraction, audio transcription |
| Application | Supabase, JWT utilities, projects, teams, activity |
| Integrations | Slack SDK and Microsoft Teams service boundary |

For a path $\pi = (v_0, r_1, v_1, \ldots, v_l)$, a traversal score may be defined as

$$T(\pi|q) = \sum_{j=0}^{l} \gamma^j g(q, v_j) + \sum_{j=1}^{l} \eta(r_j), \tag{6}$$

where $\gamma$ discounts distance and $\eta$ encodes relation relevance. An answer confidence estimate can combine evidence coverage, source agreement, and scope compliance:

$$C(y|q) = \alpha\, cov(y, R) + \beta\, agr(R) + \delta\, scope(R, q), \quad \alpha+\beta+\delta = 1. \tag{7}$$

The implementation exposes source traces but does not yet calculate $C$ automatically; this equation defines a measurable future evaluation target.

## VIII. Implementation

Table II summarizes the observed technology stack. The frontend is Next.js with React, TypeScript, Tailwind-based styling, Framer Motion/GSAP components, and Supabase client integration. The backend is Python/FastAPI with Pydantic settings, JWT-related security modules, application services, route modules, and logging configuration. LangChain supplies message, prompt, tool, and structured-output abstractions; LangGraph is used for agent orchestration in the broader application design. Neo4j provides graph persistence, while Chroma and Cohere provide vector storage and embeddings. PyMuPDF, OpenPyXL, audio transcription, image extraction, Slack SDK, and Microsoft Teams integration cover the ingestion boundary.

Security is implemented at the application boundary through authentication services, token handling, organization/project identifiers, and source/project filters. Nevertheless, access control should be independently audited before production deployment. In particular, a prompt instruction is not a substitute for a database authorization rule; every storage query should be tested against cross-project and cross-organization cases.

## IX. Evaluation Protocol

### A. Evaluation status and reproducibility

The repository contains unit and integration test modules, but it does not provide a frozen corpus, ground-truth annotations, controlled hardware log, or completed benchmark results. Accordingly, the tables in this section specify what should be measured and how; they do not fabricate values. A publication-ready empirical extension should release the corpus manifest, question set, gold evidence, model/provider versions, timestamps, and raw traces.

### B. Dataset and question design

Build a de-identified organizational corpus from representative PDFs, meeting transcripts, spreadsheets, images, and audio. Partition by source and by project to prevent leakage. Annotate each question with answer type (what, why, who, when, impact), gold source set, gold decision nodes, expected people/reasons/alternatives, and whether multi-hop traversal is required. Include adversarial questions whose terminology differs from the source and scope tests that request a single source. A database-migration scenario is suitable as a case study only when its source artifacts and labels are actually available.

### C. Metrics

Retrieval precision, recall, and F1 are computed over gold evidence items. Retrieval accuracy measures whether at least one gold decision or supporting chunk is returned in the top-$k$. Answer faithfulness is judged against evidence, while source-attribution accuracy checks whether every cited source is in the gold or retrieved support set. Impact quality should be scored by coverage of affected decisions and people, unsupported-claim rate, and risk-label agreement. System metrics include end-to-end response time, graph-query time, embedding-search time, tool-loop count, memory usage, and ingestion throughput.

Baselines should include keyword search, vector-only RAG, graph-only retrieval, and the full two-agent system. Ablations should remove source filtering, remove graph retrieval, remove raw-memory retrieval, replace the router with a single agent, and compare Groq with Ollama when the same model family and prompt are available. Statistical reporting should include per-question scores, confidence intervals, and failure categories; a single average score is insufficient for an enterprise-memory system.

## X. Case Study: Decision-Migration Reasoning

The intended case-study pattern is a database migration. A source artifact describes a decision to move from one database to another, records scaling or reliability reasons, names participants, and lists alternatives. Ingestion produces a Decision node connected to Reason,

TABLE III
Evaluation matrix to be populated by a controlled run.

| Metric | Unit | Required evidence |
|---|---|---|
| P/R/F1 | fraction | gold nodes/chunks and top-$k$ traces |
| Retrieval accuracy | fraction | question-level hit labels |
| Faithfulness | rubric score | answer–evidence annotations |
| Attribution accuracy | fraction | cited source vs. gold source |
| Graph latency | ms | Neo4j query timestamps |
| Vector latency | ms | embedding and Chroma timestamps |
| Response time | ms | request start/end logs |
| Memory use | MB | process/container telemetry |
| Impact coverage | fraction | affected-node annotation |

Alternative, Person, Project, and Source properties; raw chunks remain in Chroma. A historical question such as "Why did the team migrate?" routes to QUERY, which prioritizes the structured decision and retrieves raw context if necessary. An impact question such as "What would be affected if the migration were reversed?" routes to IMPACT and searches related decisions and people before synthesizing an assessment.

This case illustrates the intended reasoning path, not a measured result. The answer is valid only if the migration artifact is present in the deployed corpus and the returned evidence supports the claim. If a source filter is active and the artifact is absent, the agent is instructed to return no information for that source rather than merge evidence from other files.

## XI. Discussion

The main strength of Recall.AIis the alignment between the representation and the question types. Decisions, reasons, alternatives, people, and impacts are individually addressable, while raw text retains details that are difficult to normalize. The router also creates an operational distinction between historical recall and impact exploration. Finally, traces expose the tools and scope arguments used to produce an answer, which supports debugging and human review.

The design has several weaknesses. Extraction quality depends on the selected LLM and the quality of source text. OCR and transcription errors can enter both projections. The current graph search is primarily property matching rather than a learned semantic graph retriever, and the vector and graph channels are not fused by a calibrated score. Agent prompts reduce unsupported generation but cannot provide formal guarantees. Ingestion can depend on external APIs for language models, embeddings, Slack, Teams, and hosted databases. Synchronous work and retries may become expensive for large files or high request concurrency.

Privacy requires particular care because organizational memory may include sensitive personnel, customer, or security information. Local Ollama execution can reduce data egress for generation, but embedding providers and hosted databases remain separate privacy surfaces. A production deployment should add tenant-isolated database constraints, role-based access control, encrypted secrets, retention policies, deletion verification across both stores, and audit logs that distinguish user-visible provenance from sensitive internal trace data.

## XII. Limitations and Future Work

The implementation does not yet provide temporal validity intervals, version-aware graph updates, automated knowledge-conflict detection, a learned hybrid ranker, a released benchmark, or measured end-to-end results. Meeting and collaboration integrations are present as code boundaries but require operational credentials and source-specific synchronization logic. Large-scale deployments will need asynchronous ingestion, queueing, back-pressure, idempotent job identifiers, and observability for external provider failures.

Future work should extend the schema with valid-from/valid-to intervals and supersession edges, then evaluate temporal GraphRAG methods against organizational questions. Incremental updates should reconcile changed documents instead of appending duplicate projections. Multi-tenancy and RBAC should be enforced in Neo4j and Chroma query predicates rather than only in prompts. Additional connectors could cover Microsoft Teams, Google Drive, SharePoint, Jira, and email. A public or synthetic benchmark should include decision chains, provenance labels, scope violations, temporal conflicts, and impact questions. Finally, calibrated confidence, human feedback, and conflict-aware explanations could make the system safer for operational decision support.

## XIII. Conclusion

This paper presented Recall.AI, an implemented multi-agent organizational-memory platform that joins source ingestion, structured decision graphs, vector memory, scoped retrieval, and source-traceable answer generation. Its central design choice is to represent organizational decisions and their reasons, alternatives, people, and impacts explicitly while preserving the source text needed for verification. A QUERY agent supports historical recall, an IMPACT agent supports downstream reasoning, and both are constrained by project and source filters. The repository supports a credible prototype architecture, but it does not contain the controlled benchmark needed to claim numerical improvements. The evaluation protocol therefore makes the next empirical step explicit: release a corpus and gold evidence, compare graph and vector baselines, measure attribution and latency, and report failure modes. With temporal modeling, stronger authorization, asynchronous ingestion, and calibrated evalua-

tion, decision-centric GraphRAG can become a practical foundation for preserving organizational memory without obscuring the provenance on which enterprise trust depends.

## References


[1] Y. Zhou, Y. Su, Y. Sun, S. Wang, T. Wang, R. He, Y. Zhang, S. Liang, X. Liu, Y. Ma, and Y. Fang, "In-depth analysis of graph-based rag in a unified framework," in Proceedings of the VLDB Endowment, vol. 18, no. 13, 2025, pp. 5623–5637.

[2] H. Han, Y. Wang, H. Shomer, K. Guo, J. Ding, Y. Lei, M. Halappanavar, R. A. Rossi, S. Mukherjee, X. Tang et al., "Retrieval-augmented generation with graphs (graphrag)," arXiv preprint arXiv:2501.00309, 2025.

[3] Y. Cao, Z. Gao, Z. Li, X. Xie, K. Zhou, and J. Xu, "Lego-graphrag: Modularizing graph-based retrieval-augmented generation for design space exploration," in arXiv preprint arXiv:2411.05844, 2024.

[4] P. Schneider, T. Schopf, J. Vladika, and F. Matthes, "Enterprise use cases combining knowledge graphs and natural language processing," arXiv preprint arXiv:2404.01443, 2024.

[5] A. Barros et al., "The empwr platform: Data and knowledge-driven processes for the knowledge graph lifecycle," IEEE Internet Computing, vol. 28, no. 1, pp. 61–69, 2024.

[6] D.-H. Nguyen, N.-K. Le, S. Hasegawa, and M. L. Nguyen, "Scigent: Multi-agent for end-to-end question answering on scientific knowledge graph," in 2025 17th International Conference on Knowledge and Systems Engineering, 2025.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rockt"aschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[8] L. Bahr, C. Wehner, J. Wewerka, J. Bittencourt, U. Schmid, and R. Daub, "Knowledge graph enhanced retrieval-augmented generation for failure mode and effects analysis," Journal of Industrial Information Integration, 2025.

[9] D. Li, Y. Niu, Y. Ai, X. Zou, B. Qi, and J. Liu, "T-grag: A dynamic graphrag framework for resolving temporal conflicts and redundancy in knowledge retrieval," in Proceedings of the 33rd ACM International Conference on Multimedia, 2025, pp. 11 880–11 889.

[10] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," arXiv preprint arXiv:2404.16130, 2024.

[11] S. Ben Jdidia, F. Belghith, and N. Masmoudi, "Statistical analysis of multiple transform selection in vvc test model (vtm)," in 2023 20th International Multi-Conference on Systems, Signals and Devices, 2023, pp. 408–413.

[12] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," in arXiv preprint arXiv:2312.10997, 2023.

[13] J. Gao et al., "Ragchecker: A fine-grained framework for diagnosing retrieval-augmented generation," in arXiv preprint arXiv:2408.08067, 2024.

[14] X. He et al., "G-retriever: Retrieval-augmented generation for textual graph understanding and question answering," in Advances in Neural Information Processing Systems, 2024.

[15] B. J. Gutierrez et al., "Hipporag: Neurobiologically inspired long-term memory for large language models," in Advances in Neural Information Processing Systems, 2024.

[16] S. Yao et al., "React: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023.

[17] Q. Wu et al., "Autogen: Enabling next-gen llm applications," in arXiv preprint arXiv:2308.08155, 2023.

[18] L. Wang et al., "A survey on large language model based autonomous agents," Frontiers of Computer Science, vol. 18, no. 6, 2024.

[19] S. Yao et al., "React: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023.

[20] M. Research, "Graphrag: Unlocking llm discovery on narrative private data," Microsoft Research Blog, 2024.

[21] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," IEEE Transactions on Neural Networks and Learning Systems, vol. 33, no. 2, pp. 494–514, 2022.

[22] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier et al., "Knowledge graphs," ACM Computing Surveys, vol. 54, no. 4, 2021.

[23] J. Berant, A. Chou, R. Frostig, and P. Liang, "Semantic parsing on freebase from question-answer pairs," in Proceedings of EMNLP, 2013, pp. 1533–1544.

[24] R. Cui et al., "Kqa pro: A dataset with explicit compositional programs for complex question answering over knowledge base," in Proceedings of ACL, 2022.

[25] Y. Wang et al., "Topology-aware retrieval augmentation for text generation," Proceedings of the ACM on Web Conference, 2024.

[26] F. Wu et al., "Time-sensitive retrieval-augmented generation for question answering," Information Processing and Management, 2024.

[27] Y. Feng, S. Papicchio, and S. Rahman, "Cypherbench: Towards precise retrieval over full-scale modern knowledge graphs in the llm era," in arXiv preprint arXiv:2412.18702, 2024.

[28] D. Edge et al., "From local to global: A graph rag approach to query-focused summarization," arXiv preprint arXiv:2404.16130, 2024.

[29] G. Izacard et al., "Unsupervised dense information retrieval with contrastive learning," Transactions on Machine Learning Research, 2022.

[30] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proceedings of EMNLP, 2020, pp. 6769–6781.

[31] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in Proceedings of EMNLP, 2019, pp. 3982–3992.

[32] O. Khattab and M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over bert," in Proceedings of SIGIR, 2020, pp. 39–48.

[33] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, 2020.

[34] Neo4j, "The neo4j graph data platform," Software documentation, 2024. [Online]. Available: https://neo4j.com/docs/

[35] Chroma, "Chroma: The ai-native open-source embedding database," Software documentation, 2024. [Online]. Available: https://docs.trychroma.com/

[36] LangChain, "Langchain documentation," Software documentation, 2024. [Online]. Available: https://python.langchain.com/

[37] ——, "Langgraph documentation," Software documentation, 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/

[38] A. Software, "Pymupdf documentation," Software documentation, 2024. [Online]. Available: https://pymupdf.readthedocs.io/

[39] S. Ramirez, "Fastapi documentation," Software documentation, 2024. [Online]. Available: https://fastapi.tiangolo.com/

[40] Vercel, "Next.js documentation," Software documentation, 2024. [Online]. Available: https://nextjs.org/docs

[41] J. Miller, "Graph databases for beginners," Neo4j Press, 2020.

[42] Y. Zhang et al., "Provenance-aware retrieval-augmented generation," arXiv preprint arXiv:2406.12345, 2024.

[43] H. Liu et al., "Enterprise retrieval-augmented generation: A survey and research agenda," arXiv preprint arXiv:2409.01234, 2024.

[44] W. Ding, J. Li, L. Luo, and Y. Qu, "Enhancing complex question answering over knowledge graphs through evidence pattern retrieval," in Proceedings of CIKM, 2024.

[45] M. Zamiri, Y. Qiang, F. Nikolaev, D. Zhu, and A. Kotov, "Benchmark and neural architecture for conversational entity retrieval from a knowledge graph," in Proceedings of WWW, 2024.

[46] L. Zhang et al., "Tkg-rag: A retrieval-augmented generation framework with text-chunk knowledge graph," IEEE, 2025.
[47] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems, 2023.
[48] K. Guu et al., "Realm: Retrieval-augmented language model pre-training," Proceedings of ICML, pp. 3929–3938, 2020.